Learning Objectives

Learners will be able to...

- **Understand the internal workings of different Set implementations in C++, such as `std::unordered_set` and `std::set`**
- **Explore the trade-offs and performance characteristics of each Set implementation, including time and space complexity.**
- **Learn how hashing is used in `std::unordered_set` and how it affects performance and collision handling.**
- **Examine the underlying tree structure of `std::set` and understand how it maintains sorted order and provides logarithmic time operations.**

Set Implementations in C++

Implementing Set in C++

In C++ Standard Template Library (STL) provides similar functionalities through different containers such as `std::unordered_set`, `std::set`, and `std::unordered_multiset`. Each of these containers has unique characteristics and performance implications. In this section, we'll take a closer look at three primary implementations: `std::unordered_set`, `std::set`, and `std::unordered_multiset`.

std::unordered_set

- **Underlying Structure**: `std::unordered_set` is backed by a hash table.
- **Performance**: Offers average constant time performance for basic operations (insert, erase, find) assuming the hash function disperses elements evenly.
- **Ordering**: Does not guarantee any specific order of the elements.
- **Null Elements**: Since C++ does not have `null`, the equivalent is storing pointers, and `std::unordered_set` can store `nullptr` if used with pointer types.

std::set

- **Underlying Structure**: `std::set` is implemented using a self-balancing binary search tree, typically a red-black tree.
- **Performance**: Provides logarithmic time complexity for basic operations (insert, erase, find).
- **Ordering**: Maintains elements in sorted order based on their values or a custom comparator provided by the user.
- **Null Elements**: Not applicable, but can store pointers and manage `nullptr` similarly to `std::unordered_set`.

std::unordered_multiset

- **Underlying Structure**: `std::unordered_multiset` is a hash table that allows duplicate elements.
- **Performance**: Similar to `std::unordered_set`, offering average constant time performance for basic operations.
- **Ordering**: Does not guarantee any order of elements.
- **Null Elements**: Like `std::unordered_set`, it can store `nullptr` when using pointer types.

Over the following pages, we will delve deeper into each of these implementations, exploring their internal workings, advantages, and use cases. This will help you understand when to use each implementation and

how to leverage their unique characteristics in your C++ applications.

std::unordered_set

The `std::unordered_set` Implementation

`std::unordered_set` is one of the most commonly used set-like containers in C++. It is backed by a hash table and provides average constant-time performance for basic operations, assuming the hash function disperses the elements properly.

Internal Working

- **Hash Table**: `std::unordered_set` uses a hash table internally to store its elements. Each element is hashed to determine its position in the table.
- **Hashing**: When an element is added to the `std::unordered_set`, it is hashed using its hash function to determine its bucket in the hash table.
- **Collision Handling**: If two elements have the same hash value (a collision), they are stored in a linked list or another structure (depending on implementation) at that position. Modern implementations may use advanced techniques like buckets or trees for performance optimization, depending on the number of collisions.

Performance Characteristics

- **Time Complexity**: Average-case time complexity for insert, erase, and find operations is $O(1)$. In the worst case, due to collisions, the time complexity can degrade to $O(n)$.
- **Space Complexity**: The space complexity of a `std::unordered_set` is $O(n)$, where n is the number of elements in the set.

When to Use `std::unordered_set`

- **Uniqueness**: When you need to ensure that a collection contains no duplicate elements.
- **Fast Access**: When you need fast access to elements and the order of elements is not important.
- **Null Values**: Not applicable in C++, but you can store `nullptr` when using pointer types.

Using `std::unordered_set`

We have seen the `std::unordered_set` in a previous example. Create an `std::unordered_set` to manage a collection of unique strings.

```cpp
#include <iostream>
#include <unordered_set>

int main() {
    std::unordered_set<std::string> names;
    names.insert("Alice");
    names.insert("Bob");
    names.insert("Charlie");
    names.insert("Alice"); // Duplicate element, not added

    std::cout << "Names: ";
    for (const auto& name : names) {
        std::cout << name << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

You should see output similar to:

```
Names: Bob Alice Charlie
```

challenge

Try this variation

Add `nullptr` to the set using a set of pointer types.

```cpp
#include <iostream>
#include <unordered_set>

int main() {
    std::unordered_set<std::string*> names;
    std::string alice = "Alice";
    std::string bob = "Bob";
    std::string charlie = "Charlie";

    names.insert(&alice);
    names.insert(&bob);
    names.insert(&charlie);
    names.insert(nullptr);  // Adding nullptr to the set

    std::cout << "Names: ";
    for (const auto& namePtr : names) {
        if (namePtr != nullptr)
            std::cout << *namePtr << " ";
        else
            std::cout << "nullptr ";
    }
    std::cout << std::endl;

    return 0;
}
```

▼ **What is happening?**

> When using a `std::unordered_set` to implement the set ADT, you can add `nullptr` as an element.

In this example, we create an `std::unordered_set` to store a collection of names. When we try to add the duplicate element “Alice” again, it is not added to the `set`, demonstrating the uniqueness property of `std::unordered_set`.

In the next section, we will explore `std::set` and its internal workings, along with its performance characteristics and use cases.

std::set

The `std::set` Implementation

`std::set` is a widely used container in C++ that provides an efficient way to store elements in sorted order. It is backed by a self-balancing binary search tree, typically a Red-Black tree. Unlike `std::unordered_set`, `std::set` maintains the elements in sorted order based on their values or a custom comparator.

Internal Working

- **Red-Black Tree**: `std::set` uses a Red-Black tree to store elements, ensuring that the tree remains balanced, providing logarithmic time performance for basic operations.
- **Sorting**: Elements in a `std::set` are sorted according to their values or by a specified comparator function.
- **No Duplicates**: Like other set implementations, `std::set` does not allow duplicate elements. If an element is inserted that is equal to an existing element in the set, it is not added.

Performance Characteristics

- **Time Complexity**: The time complexity for insert, erase, and find operations is $O(logn)$, where n is the number of elements in the set.
- **Space Complexity**: The space complexity of a `std::set` is $O(n)$.

When to Use `std::set`

- **Sorted Elements**: Use `std::set` when you need to maintain the elements in sorted order.
- **Range Queries**: `std::set` is useful for performing range queries, such as finding elements within a specific range or retrieving elements greater than or less than a certain value.
- **No Null Values**: Like in C++, `std::set` cannot store `nullptr` unless working with pointer types.

Using `std::set`

Using `std::set` works similarly to `std::unordered_set`. Notice how we are inserting values in a random order, but the output will be in numerical order when printed. Add the following `main` function to the program:

```cpp
#include <iostream>
#include <set>

int main() {
    std::set<int> numbers;
    numbers.insert(3);
    numbers.insert(1);
    numbers.insert(2);

    std::cout << "Sorted numbers: ";
    for (const auto& number : numbers) {
        std::cout << number << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

You should see the following output:

```
Sorted numbers: 1 2 3
```

challenge

Try this variation

Try adding `nullptr` to a set of pointer types.

```
int main() {
    std::set<int*> numbers;
    int a = 3, b = 1, c = 2;
    numbers.insert(&a);
    numbers.insert(&b);
    numbers.insert(&c);
    numbers.insert(nullptr);  // Adding nullptr to the set

    std::cout << "Numbers: ";
    for (const auto& number : numbers) {
        if (number != nullptr)
            std::cout << *number << " ";
        else
            std::cout << "nullptr ";
    }
    std::cout << std::endl;

    return 0;
}
```

▼ **What is happening?**

- In C++, you can add nullptr to a `std::set<int*>` because the set compares pointer values (addresses) without dereferencing them. Since nullptr is a valid pointer (just pointing to nothing), it gets inserted as the smallest value.
- In Java, null cannot be compared to integers or any other objects because it lacks a meaningful order, causing the program to throw a NullPointerException

In this example, we create an `std::set` to store a collection of integers. When we add the elements, they are automatically sorted in ascending order, demonstrating the sorting property of `std::set`.

Custom Sorting with std::set

When storing user-defined classes in an `std::set`, there is no natural sorting order. Instead, we can provide a custom comparator. Let's create the `Person` class. A `Person` object has a `name` and an `age`. We will override the `operator<<` to help with printing. Place the `Person` class above `main` in the program.

```
class Person {
public:
    int age;
    std::string name;

    Person(int age, const std::string& name) : age(age),
        name(name) {}

    friend std::ostream& operator<<(std::ostream& os, const
        Person& p) {
        os << p.name << " - " << p.age;
        return os;
    }
};
```

Next, we create a custom `AgeComparator` that compares two Person objects based on their age, sorting them in descending order.

```
struct AgeComparator {
    bool operator()(const Person& p1, const Person& p2) const {
        return p1.age > p2.age;  // Compare ages in descending
        order
    }
};
```

Now, in the `main` function, create a set of Person objects with a std::set and `AgeComparator`. Add `four` people with different `ages` and `names`, then `print` the set.

```
int main() {
    std::set<Person, AgeComparator> people;
    people.emplace(25, "Alice");
    people.emplace(30, "Bob");
    people.emplace(20, "Charlie");
    people.emplace(35, "David");

    std::cout << "Sorted people: ";
    for (const auto& person : people) {
        std::cout << person << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

▼ Code

> Your code should look like this:

```
#include <iostream>
#include <set>

class Person {
public:
    int age;
    std::string name;

    Person(int age, const std::string& name) : age(age),
        name(name) {}

    friend std::ostream& operator<<(std::ostream& os, const
        Person& p) {
        os << p.name << " - " << p.age;
        return os;
    }
};

struct AgeComparator {
    bool operator()(const Person& p1, const Person& p2)
        const {
        return p1.age > p2.age;  // Compare ages in
        descending order
    }
};

int main() {
    std::set<Person, AgeComparator> people;
    people.emplace(25, "Alice");
    people.emplace(30, "Bob");
    people.emplace(20, "Charlie");
    people.emplace(35, "David");

    std::cout << "Sorted people: ";
    for (const auto& person : people) {
        std::cout << person << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

You should see the following output:

```
Sorted people: David - 35 Bob - 30 Alice - 25 Charlie - 20
```

Try this variation

Modify the code so that Person objects added to the set will be sorted in ascending order by age.

▼ **Solution**

> Change the AgeComparator to compare in ascending order:

```
struct AgeComparator {
    bool operator()(const Person& p1, const Person& p2)
        const {
        return p1.age < p2.age;
    }
};
```

In this example, we demonstrated how to use std::set to store a collection of objects with a custom sorting order. You can specify your own comparator to sort objects that do not have a natural ordering.

In the next section, we will explore `std::unordered_multiset` and its unique features, including how it allows duplicate elements and provides constant-time performance for basic operations.

unordered_set with list

The `std::unordered_set` with `std::list` Implementation

In C++ , we can simulate the behavior of a `LinkedHashSet` by combining `std::unordered_set` and `std::list`. While C++ does not have a direct equivalent to Java's `LinkedHashSet`, this combination can be used to maintain insertion order while ensuring unique elements.

Internal Working

- **Hash Table with Linked List**: `std::unordered_set` uses a hash table internally to store its elements, similar to `HashSet` in Java. We can use `std::list` to maintain the insertion order of elements.
- **Ordering**: Elements in this combination are iterated in the order in which they were inserted using the `std::list`.
- **Performance**: The performance of basic operations (insert, remove, find) is similar to `std::unordered_set`, but with some added complexity for maintaining insertion order via the list.

Performance Characteristics

- **Time Complexity**: Average-case time complexity for insert, remove, and find operations is $O(1)$. The worst-case time complexity can degrade to $O(n)$ due to hash collisions, similar to `std::unordered_set`.
- **Space Complexity**: The space complexity of this combination is higher than that of a simple `std::unordered_set` due to the additional overhead of maintaining a `std::list`.

When to Use this Combination

- **Insertion Order**: Use this approach when you need to maintain the order in which elements were inserted into the set.
- **No Duplicates**: Like other set implementations, this combination does not allow duplicate elements.
- **Performance**: If the performance impact of maintaining insertion order is acceptable for your application, this combination can be a good choice.

Using `std::unordered_set` with `std::list`

In this example, we use `std::unordered_set` to ensure uniqueness and `std::list` to maintain insertion order. Notice the order in which elements are added to the set. The output should maintain this order.

```cpp
#include <iostream>
#include <unordered_set>
#include <list>

int main() {
    std::unordered_set<std::string> set;
    std::list<std::string> order;

    auto insert = [&](const std::string& name) {
        if (set.insert(name).second) {
            order.push_back(name); // Insert only if not already
        present
        }
    };

    insert("Charlie");
    insert("Alice");
    insert("Bob");

    std::cout << "Names in insertion order: ";
    for (const auto& name : order) {
        std::cout << name << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

You should see the following output:

```
Names in insertion order: Charlie Alice Bob
```

challenge

Try this variation

Add nullptr (or null equivalent in C++) to the list.

```
#include <iostream>
#include <unordered_set>
#include <list>

int main() {
    std::unordered_set<std::string> set;
    std::list<std::string> order;

    auto insert = [&](const std::string& name) {
        if (set.insert(name).second) {
            order.push_back(name);
        }
    };

    insert("Alice");
    insert("Bob");
    insert("Charlie");
    insert(""); // In C++, an empty string is used instead
of null

    std::cout << "Names: ";
    for (const auto& name : order) {
        if (name.empty()) {
            std::cout << "nullptr ";
        } else {
            std::cout << name << " ";
        }
    }
    std::cout << std::endl;

    return 0;
}
```

In this example, we simulate adding a null element by inserting an empty string into the set.

Sets and Thread Safety

Concurrency and Sets

When working with sets in a multi-threaded environment, it's crucial to be aware of potential issues that can arise from concurrent access. The implementations of the set Abstract Data Type (ADT) discussed in this module are not thread-safe. This means that when multiple threads interact with the same set simultaneously, it can lead to unexpected behavior. While this topic may be beyond the scope of this course, understanding thread safety is important.

In the following example, two threads access the set concurrently. The first thread iterates over the range from 1 to 1,000,000, adding each number to the set. The second thread iterates over the elements in the set. The second loop only performs an iteration without modifying the set.

Here's how the code looks:

```
std::set<int> sharedSet;

void addElements() {
    for (int i = 1; i <= 1000000; i++) {
        sharedSet.insert(i); // Unsafe access
    }
    std::cout << "Thread 1 finished adding elements." <<
        std::endl;
}

void iterateSet() {
    for (const auto& value : sharedSet) { // Unsafe access
        // Do something with the value
        // ...
    }
    std::cout << "Thread 2 finished iterating over the set." <<
        std::endl;
}

int main() {
    std::thread t1(addElements);
    std::thread t2(iterateSet);

    t1.join();
    t2.join();

    return 0;
}
```

Running this code may show the error or result in undefined behavior due to concurrent modification of the set, as there is no synchronization mechanism in place.

Synchronizing

To avoid these problems, we can use a `mutex` to synchronize access to the set. This ensures that thread operations are coordinated, preventing concurrent access issues.

Here's the modified code that ensures thread safety:

```
std::set<int> sharedSet;
std::mutex mtx;

void addElements() {
    for (int i = 1; i <= 1000000; i++) {
        std::lock_guard<std::mutex> lock(mtx); // Lock for
        thread safety
        sharedSet.insert(i);
    }
    std::cout << "Thread 1 finished adding elements." <<
        std::endl;
}

void iterateSet() {
    std::lock_guard<std::mutex> lock(mtx); // Lock for thread
        safety
    for (const auto& value : sharedSet) {
        // Do something with the value
        // ...
    }
    std::cout << "Thread 2 finished iterating over the set." <<
        std::endl;
}

int main() {
    std::thread t1(addElements);
    std::thread t2(iterateSet);

    t1.join();
    t2.join();

    return 0;
}
```

Using a `mutex` to synchronize access introduces some overhead, which can affect overall performance. However, it is essential for ensuring data integrity in `multi-threaded` applications.

When running the synchronized version, you should see output similar to:

```
Thread 2 finished iterating over the set.
Thread 1 finished adding elements
```

Challenges & Considerations with Sets

Things to Consider When Using Sets

While sets provide powerful tools for handling collections of unique elements, there are several challenges and considerations to keep in mind when working with sets in C++.

Applications of Sets

- **Duplicate Removal**: Sets can be used to remove duplicate elements from a collection, which is useful in data processing and analysis.
- **Set Intersection**: Set intersection can be used to find common elements between two collections, which is useful in areas like collaborative filtering and recommendation systems.
- **Access Control**: Sets can be used to manage access control in software systems, where each set represents a group of permissions or roles.
- **Graph Theory**: In graph theory, sets are used to represent vertices and edges in a graph, which is fundamental in algorithms like shortest path and network flow.

Performance Considerations

- **std::unordered_set**: Offers average constant-time performance for basic operations like `insert`, `erase`, and `find`. However, the performance heavily depends on the quality of the hash function.
- **std::set**: Provides logarithmic time performance for basic operations. It is slower than `std::unordered_set` for operations like `insert` and `erase` but maintains elements in a sorted order.
- **std::multiset**: Similar to `std::set`, but allows duplicate elements. Useful when you need to maintain sorted order and allow duplicates.

Memory Overhead

- **Higher Overhead**: Sets, particularly `std::unordered_set`, have a higher memory overhead compared to arrays or vectors due to the underlying data structures used for ensuring uniqueness and supporting fast access.

Choosing the Right Set Implementation

The choice of set implementation (`std::unordered_set`, `std::set`, or `std::multiset`) depends on the specific requirements of your application:

- **Sorting**: Use `std::set` or `std::multiset` if sorting is required.

- **Duplicates**: Use `std::multiset` if duplicates are allowed.
- **Performance**: Use `std::unordered_set` if performance for `insert`, `erase`, and `find` is the priority and order doesn't matter.

Handling Null-Equivalent Values

- **Pointer Types**: C++ set implementations do not directly support null values, but for pointer types, you can store `nullptr` in `std::unordered_set` or `std::set`.
- **Value Types**: For value types (non-pointer), you might need to use `std::optional` or define a sentinel value to represent "null" in your specific use case, as C++ doesn't have a native `null` value for non-pointer types.

Thread Safety

- **Not Thread-Safe**: The standard set implementations in C++ (`std::unordered_set`, `std::set`, `std::multiset`) are not thread-safe. If multiple threads access a set concurrently and at least one of the threads modifies the set, it must be synchronized externally using `std::mutex` or other synchronization mechanisms.

Formative Assessment 1

Formative Assessment 2